공학 학사 학위논문

DCT-Page: Frequency-Domain Page Proxies for Sparse KV Decoding

2026년 6월

서울대학교 공과대학

전기·정보공학부

김 윤 곤

DCT-Page: Frequency-Domain Page Proxies for Sparse KV Decoding

지도교수 김재준

이 논문을 공학 학사학위 논문으로 제출함.

서울대학교 공과대학

전기·정보공학부

김 윤 곤

김윤곤의 학사 학위 논문을 인준함.

2026년 6월

지도교수 김 재 준 (인)

ABSTRACT

Decoder transformer language models at long context spend most of their decode-time budget reading a growing key–value cache from memory, putting per-step latency against the HBM-bandwidth ceiling. Sparse-attention methods sidestep this bottleneck by reading only a small subset of past tokens, but existing page-based selectors rely on coarse per-page statistics (Quest's per-channel min/max [1]) or on learned gates that must be trained per model (SeerAttention-R [2]), and the cluster-based alternative (Multipole Attention [3]) achieves higher accuracy at the cost of a data-dependent decode path whose irregular memory-access pattern makes it multiples slower in practice than the page-based methods.

This thesis introduces DCT-Page, a training-free decode-time sparse attention method built on a single structural observation: contiguous pages of attention keys live overwhelmingly in their lowest few discrete-cosine-transform bins, with the first four bins of a 32-token page covering on average $\approx 85\%$ of the page energy. DCT-Page exploits this by replacing each page with a short DCT-lowpass-IDCT proxy — a single precomputed linear projection applied as a small matrix multiply per closed page — and scoring pages by the query's inner product against the proxy. The top-k pages are read at full precision and the rest are dropped. Scoring and selection run as three fused GPU kernels on top of upstream FlashInfer's paged-attention primitive [4], and the entire decode loop is CUDA-graph capturable.

On RULER at 32K context, DCT-Page leads the best page-based sparse baseline by +5.16 and +6.40 points on Llama-3.1-8B-Instruct and Qwen3-8B respectively, stays within 0.07 points of Full-KV on LongBench v1, and is the only sparse method whose accuracy does not collapse on the longest LongBench v2 context bucket. An attention-mass-recall analysis traces this advantage to its source: at the same per-step budget DCT-Page reaches $\approx 84\%$ of the mass-topk oracle's achievable middle-page recall against the page-based baseline's 65.6%. On a single A6000 GPU at 128K context, DCT-Page is 5.65× faster than Full-KV in the attention kernel and 1.70× faster end-to-end.

Keywords: Long-context Language Model, Sparse Attention, KV Cache, Discrete Cosine Transform, Page Attention

Contents

List of Tables

List of Figures

Chapter 1 Introduction

1.1 Motivation

Large language models are increasingly deployed on inputs of hundreds of thousands of tokens, from long-form documents and retrieved-context question answering to multi-file code understanding and extended chat histories. These workloads spend most of their wall-clock budget in *autoregressive decoding*, where the model emits one token at a time and each new token must attend to the entire prior key–value (KV) cache. As Section 2.1 reviews in more detail, this puts decode latency squarely against the HBM-bandwidth ceiling: with an 8B-parameter model in `bfloat16` and a 128K context, the KV cache occupies tens of gigabytes and the per-step attention takes longer than every other layer combined.

The natural way out is to make the per-step attention sparse: identify the small subset of past tokens (or contiguous pages of tokens) that the dense softmax would have weighted heavily and read only those, leaving the rest in memory unused. A body of recent work has explored this idea along two structurally different paradigms. *Page-based* methods such as Quest [1], SeerAttention-R [2], and InfLLM [5] partition the cache into fixed-size pages and score each page against the current query. *Cluster-based* methods such as Multipole Attention [3] instead group the cache into learned k-means clusters and attend through cluster representatives. Page-based selection is fast and amenable to CUDA-graph capture, but accuracy is bounded by how informative the per-page summary statistics are: Quest's per-channel min/max, for example, discards intra-page structure and collapses on the longest LongBench v2 context bucket. Cluster-based selection achieves higher accuracy on synthetic benchmarks but its data-dependent decode path incurs irregular memory accesses — centroids are recomputed during generation and selected clusters are scattered across the cache — and is multiples slower than the page-based alternatives at the same selected-token budget.

This thesis asks whether a single, training-free choice of per-page summary — one that captures intra-page structure but is cheap enough to keep the page-based throughput — can

close most of the accuracy gap to cluster-based methods without giving up their hardware-friendly decode path.

1.2 Key Insight: Attention Keys are Low-Frequency

The starting point of DCT-Page is a structural observation about the key vectors that flow through the attention mechanism: viewed as a signal along the sequence axis, a contiguous page of keys lives overwhelmingly in its lowest few frequency bins. Concretely, taking P=32-token pages of keys from any single (layer, head) pair on a RULER NIAH-MultiKey 3 trace at 32K context and applying an orthonormal type-II discrete cosine transform along the sequence axis, the first four DCT bins capture on average $\approx 85\%$ of the total per-page key energy across layers, and the first eight reach $\approx 90\%$. The high-frequency tail is essentially content-free.

This concentration has a direct algorithmic consequence. The score that determines whether a page should be selected for full attention is the inner product of the query against (some summary of) the keys inside the page. If the page's energy lives in a low-frequency subspace, then projecting the page onto its leading DCT coefficients and reconstructing back into the token domain — a *DCT-lowpass-IDCT* proxy — yields a short sequence of representative tokens that, by Parseval's theorem, retains nearly all of the inner product the query could have had with the full page. The proxy is a single fixed linear projection $M \in \mathbb{R}^{C \times P}$ that depends only on the page size P and the proxy length C, so it can be precomputed once and applied as a small matrix multiply per closed page at decode time.

DCT-Page is the system built on this observation. It divides the KV cache into fixed-size pages, summarizes each page with a DCT-lowpass-IDCT proxy, scores pages by the query's inner product against the proxy tokens, and reads only the top-k pages at full precision. The proxy is training-free, the page selection runs as a short pipeline of fused GPU kernels on top of an existing paged-attention runtime, and the entire decode loop is captured into a CUDA graph for production-grade throughput.

1.3 Contributions

This thesis makes four contributions:

1. **A training-free, frequency-domain page proxy.** Section 3.3 introduces the DCT-lowpass-IDCT projection used as a page summary: each closed page is replaced by a short sequence of representative tokens obtained from a single precomputed linear map that depends only on the page size and the proxy length. No model weights are added, no fine-tuning is required, and the same projection applies unchanged across models, layers, and heads.

2. **Page-based throughput at cluster-grade accuracy.** By encoding the intra-page key distribution rather than collapsing it to per-channel min/max, DCT-Page closes most of the accuracy gap that page-based methods have historically paid for their hardware-friendliness — while keeping their fixed, contiguous decode-time memory layout. In particular, DCT-Page is the only page-based method in our evaluation whose accuracy does not collapse on the longest LongBench v2 context bucket.

3. **A hardware-friendly decode integration.** Section 3.5 composes the proxy update, page scoring, and top-k/KV-assembly stages as three fused GPU kernels on top of upstream FlashInfer's paged-attention primitive [4], using a virtual-batching scheme to feed per-(batch, kv-head) page indices into the unmodified upstream kernel. The entire decode loop — scoring, selection, and attention — is CUDA-graph capturable, so DCT-Page inherits production-grade paged-attention throughput rather than competing with it.

4. **A principled account of why frequency-domain summaries suffice.** The structural observation that drives DCT-Page — $\approx 85\%$ of per-page key energy in the first four of thirty-two DCT bins — is sharpened in Section 6.1 into an attention-mass-recall measurement on dense reference traces, which connects the spectral concentration of keys to the softmax mass actually recovered at decode time. The same lens explains, mechanistically rather than empirically, where page-based scorers without intra-page resolution break down.

Chapter 2 Background and Related Work

2.1 Autoregressive Decoding and the KV-Cache Bottleneck

Decoder transformers [6] generate text autoregressively: at decode step t, the model conditions on every previously emitted token and produces a single new token. The attention mechanism at each layer computes, for the current query $q_t \in \mathbb{R}^d$ and the cumulative keys $K_{\le t} \in \mathbb{R}^{t\times d}$ and values $V_{\le t} \in \mathbb{R}^{t\times d}$,

$$\mathrm{Attn}(q_t, K_{\le t}, V_{\le t}) \;=\; \mathrm{softmax}\big(q_t K_{\le t}^{\top}/\sqrt{d}\big)\, V_{\le t}. \tag{2.1}$$

To avoid recomputing the past keys and values at every step, modern inference runtimes maintain a *KV cache* that grows by one row per decode step. Each step then performs an inner product between q_t and the entire cached $K_{\le t}$, followed by a softmax-weighted read of $V_{\le t}$.

The cost of Eq. (2.1) per decode step is dominated by the memory traffic of reading the cache. At context length L, the per-step attention reads $O(L \cdot d)$ entries of K and V from GPU global memory, and the arithmetic intensity of the operation (a single small matrix–vector product per head) is far below the compute-to-memory ratio of modern accelerators. As a result, decode latency scales approximately linearly with L and is bottlenecked by HBM bandwidth rather than by FLOPs. At $L = 128$K with an 8B-parameter model in `bfloat16`, the KV cache alone can occupy more than half of the device memory budget, and the per-step attention can take longer than all other layer operations combined.

Two structural observations sit behind every sparse-attention proposal of the past two years. First, even at long context the dense softmax in Eq. (2.1) concentrates most of its mass on a small subset of past tokens; the rest contributes negligible energy and could in principle be skipped without changing the output. Second, prefill — the one-pass attention over the prompt that populates the cache — is compute-bound rather than memory-bound. Concretely, prefill on an L-token prompt computes a $Q[L\times d]\cdot K^{\top}[d\times L]$ matrix–matrix product (arithmetic intensity $O(L)$ FLOP per byte loaded), whereas decode computes a $q[1\times d]\cdot K^{\top}[d\times L]$ matrix–vector product (arithmetic intensity $O(1)$, independent of L). For any L on the order of a thousand

or more, prefill is therefore in the compute-bound regime and modifying it offers little speedup, while decode is squarely memory-bound. Sparse-attention systems — DCT-Page included — therefore typically leave prefill unchanged and intervene only at decode time, replacing the full read of $K_{\leq t}, V_{\leq t}$ with some compressed or filtered surrogate.

2.2 Sparse and Compressed KV Attention

The methods DCT-Page is compared against differ along two structural axes: (i) how they *group* the KV cache into selectable units — fixed-size pages or learned clusters — and (ii) how they *score* each unit against the current query. The first three methods below share the page-based paradigm with DCT-Page; Multipole sits in the cluster-based paradigm and is discussed last.

2.2.1 Quest

Quest [1] partitions the KV cache into fixed-size pages and, for each page, stores a pair of per-channel summary statistics: the elementwise minimum and maximum of the keys inside the page. At decode time the query is scored against every page by an *approximate-max inner product*. For each channel $c \in \{1, \ldots, d\}$, the per-channel contribution is taken to be the larger of $q[c] \cdot \min_p[c]$ and $q[c] \cdot \max_p[c]$ — since the true key value $k[c]$ for any token inside the page lies in $[\min_p[c], \max_p[c]]$, this is a tight upper bound on $q[c] \cdot k[c]$ — achievable when some token in the page attains the channel extreme — and selecting between $\min_p[c]$ and $\max_p[c]$ requires only the per-channel sign of $q[c]$ ($\max_p[c]$ if $q[c] \geq 0$, $\min_p[c]$ otherwise). The page score is the sum of these per-channel upper bounds across the d channels, which is itself an upper bound on $\max_{k \in \text{page}} q^\top k$. This bound is cheap (two scalars per channel) and position-agnostic. Quest then attends to the top-k pages by this score and ignores the rest.

The per-page metadata is small — two scalars per channel rather than a learned proxy — and the kernel cost of scoring is low, which makes Quest fast at moderate context length. However, the min/max abstraction discards *intra-page* structure entirely: it cannot distinguish a page in which a few channels happen to span a wide range from a page in which one specific token has a large inner product with the query. Section 6.2.3 returns to this point empirically.

2.2.2 SeerAttention-R

SeerAttention-R [2] adds a small learned *gate* to each decoder layer that, at decode time, predicts per-block importance scores from the current query. The gate is trained on long-context data with a supervision signal derived from the dense attention distribution, so that its block scores correlate with the softmax mass that the dense model would have placed on each block. At inference, the gates are evaluated at every layer and the top-k blocks are kept under a fixed token budget.

The learned-gate approach can be very accurate at a given budget because it is explicitly optimized for the selection task, but it imposes two practical constraints. First, a model-specific checkpoint must be released for every base model; in our experiments we use the Qwen3-family gates released by the authors, since no equivalent Llama-3 gates are available, and SeerAttention-R is therefore reported on Qwen3-8B only. Second, the gate adds a small but non-zero per-layer parameter count and forward-time cost, which slightly reduces its ceiling on raw decode throughput.

2.2.3 InfLLM

InfLLM [5] approaches long-context decoding from a retrieval perspective. The past KV cache is broken into blocks; each block is summarized by a small set of *representative tokens* chosen by an upstream-defined importance heuristic; and at decode time the current query is matched against every block's representatives to produce a relevance score. InfLLM additionally keeps an initial sink region (the first n_{init} tokens) and a local sliding window, attending in full to both regardless of the retrieval result. Only the top-ranked blocks outside the sink and local zones are read at full precision.

The retrieval framing makes InfLLM agnostic to context length in principle, but it pays for this flexibility in implementation complexity: the block representatives must be selected and indexed carefully, and the published code path currently targets the Llama-3 family only. We accordingly report InfLLM on Llama-3.1-8B-Instruct alone.

2.2.4 Multipole Attention

Multipole Attention [3] departs from the fixed-page paradigm and instead clusters keys with hierarchical k-means. At decode time the query attends through cluster centroids: each cluster contributes a single representative key/value pair, and clusters whose centroids score highly are then expanded to their constituent keys for the final attention. This is the attention analogue of the fast-multipole-method approximation used in n-body simulation.

Cluster-based selection has two strengths and one structural weakness. Its strengths are that the cluster representatives carry within-page structure (a centroid is a positionless but distribution-aware summary) and that the hierarchical structure allows variable-width effective receptive fields per layer. Empirically this yields the highest long-context accuracy among the methods we evaluate (Section 5.4). Its weakness is that clusters are *data-dependent and recomputed during generation*: the centroids must be re-estimated as the KV cache grows, and the keys belonging to a selected cluster are scattered (non-contiguous) in memory. The per-step computation therefore has an irregular memory-access pattern that does not amortize across decode steps the way a fixed page layout does, and the decode-time wall clock is substantially slower than the page-based methods at the same selected-token budget. This is the source of the accuracy–throughput Pareto trade-off that Section 5.4 reports in detail.

2.3 Frequency-Domain Signal Compression

The DCT proxy of Section 3.3 is a low-rank linear projection of a key sequence onto its leading discrete-cosine coefficients. We collect here the small set of background facts that make this projection well-behaved for our application.

Discrete cosine transform. The type-II DCT used in Eq. (3.3) is an orthonormal linear transform that asymptotically diagonalizes the second-order autocorrelation matrix of a stationary Markov-1 signal — this is the classical motivation for its dominance in image and audio codecs (e.g. JPEG) [7]. Orthonormality means that energy is preserved under DCT, and Parseval's theorem applies coefficient by coefficient: discarding the higher-frequency bins reduces the sequence's ℓ_2 norm by exactly the energy of those bins. Whether the resulting low-rank

reconstruction is a good summary of the original signal therefore reduces to whether the signal's energy is in fact concentrated in the leading bins, which is the empirical question raised in Section 3.2.

Lowpass-IDCT as a linear projection. Concatenating a length-P DCT, a truncation that keeps the first C coefficients, and a length-C inverse DCT yields a single $C \times P$ matrix that can be precomputed once and applied as a small dense matrix multiply at runtime. This is the operator $M \in \mathbb{R}^{C \times P}$ of Eq. (3.4). All of the mass-recall and accuracy results in Chapter 5 use this single fixed M; nothing about the proxy is learned.

Concurrent frequency-domain work. The closest contemporary system is FreqKV [8], which uses a DCT-based representation to compress the KV cache for context-window extension rather than for sparse decoding. FreqKV operates in the prefill stage and modifies the cache layout itself, whereas DCT-Page leaves the cache unchanged and uses DCT only as a scoring proxy during decode. The two systems are complementary — one could in principle combine FreqKV's frequency-domain prefill compression with DCT-Page's frequency-domain decode-time scoring — but in this thesis we restrict attention to the decode-time use of DCT.

Chapter 3 Method: DCT-Page

DCT-Page modifies only the *decode* path of attention. During prefill the model runs unchanged, producing an ordinary key–value (KV) cache. At decode time the cache is reinterpreted as a sequence of fixed-size pages, each page is summarized by a short low-frequency proxy, and the query attends to a constant-size subset: the sink, the top-k pages by proxy score, and a small recent window. The remaining pages are skipped entirely. This chapter describes the layout, the proxy, the selection rule, and the fused kernels that turn the resulting sparsity into measured throughput.

3.1 Decode-Time Page Layout

Let L denote the current KV-cache length and P the page size (default P=32). Let S and R be the number of sink and full recent pages, respectively. The cache is partitioned as

$$\underbrace{[0,\ SP)}_{\text{sink}} \ \Big\|\ \underbrace{[SP,\ SP{+}NP)}_{\text{middle pages}} \ \Big\|\ \underbrace{[SP{+}NP,\ L)}_{\text{recent}}, \tag{3.1}$$

where

$$N \ = \ \left\lfloor \frac{L - SP - RP}{P} \right\rfloor \tag{3.2}$$

is the number of *middle* pages, i.e. the pageable region from which DCT-Page selects. The recent zone consists of R full pages plus a partial *open* page that absorbs the most recent $0, \ldots, P{-}1$ tokens; the open page is always attended in full. Figure 3.1 shows the layout for S=1, R=4, the defaults used throughout this thesis.

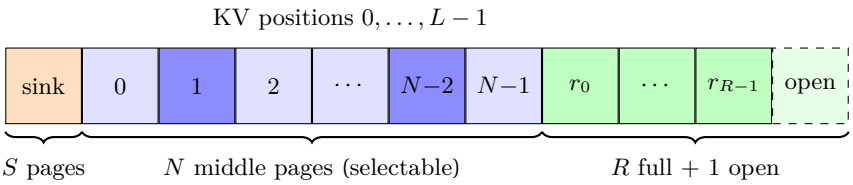

Figure 3.1: Decode-time KV layout. The sink and recent zones are always attended; only the N middle pages enter the top-k selection. Darker blue cells indicate two example selected pages.

Sink and recent zones. The sink absorbs the initial-token attention sinks observed in autoregressive decoder transformers and is kept verbatim to preserve position-aware behaviors learned during pretraining. The recent window protects the most local context, which dominates the output for generation tasks and is cheap to keep at full resolution.

Page granularity. The page size is the fundamental trade-off knob. A smaller P gives finer selection but raises bookkeeping cost (more pages, more scores, more top-k work). A larger P amortizes overhead but loses selectivity within the page. We use P=32 throughout, which is also the granularity at which contemporary sparse-attention systems operate (Quest, SeerAttention-R, Multipole).

Fallback below short context. When L is short, the savings from sparsification are dominated by the bookkeeping overhead of scoring and assembly. DCT-Page therefore falls back to standard dense decode attention when $L < L_{\min}$, with $L_{\min}$=8192 tokens by default.

3.2 Key Insight: Low-Frequency Structure of Attention Keys

The starting point for the rest of this chapter is a single empirical observation about decoder-transformer key vectors: viewed as a signal along the sequence axis, they are overwhelmingly low-frequency.

Concretely, take a page of P=32 consecutive keys $K_p \in \mathbb{R}^{P\times d}$ from a single (batch, layer, head) triple and apply the orthonormal type-II discrete cosine transform along the sequence axis,

$$\hat{K}_p[m,c] \;=\; \sqrt{\tfrac{2}{P}}\,\alpha_m \sum_{n=0}^{P-1} K_p[n,c]\,\cos\!\Big(\tfrac{\pi(2n+1)m}{2P}\Big), \quad m=0,\dots,P{-}1, \tag{3.3}$$

with $\alpha_0 = 1/\sqrt{2}$ and $\alpha_m = 1$ for $m \geq 1$. Here $\hat{K}_p \in \mathbb{R}^{P\times d}$ is the frequency-domain representation of the page: row index m runs over DCT bins from low (m=0, the DC term) to high frequency, and each column c holds the spectrum of one of the d feature channels. By Parseval's theorem (DCT is orthonormal), the total energy of K_p equals the sum of squared coefficients $\sum_m \|\hat{K}_p[m,:]\|^2$. On a RULER NIAH-MultiKey 3 trace at 32K context with Llama-3.1-8B-Instruct, the first four DCT bins ($m = 0,1,2,3$) capture, on average, $\approx 85\%$ of the total per-page key energy

across layers, and the first eight bins reach $\approx 90\%$; the remaining high-frequency tail contributes negligibly (Figure 3.2).

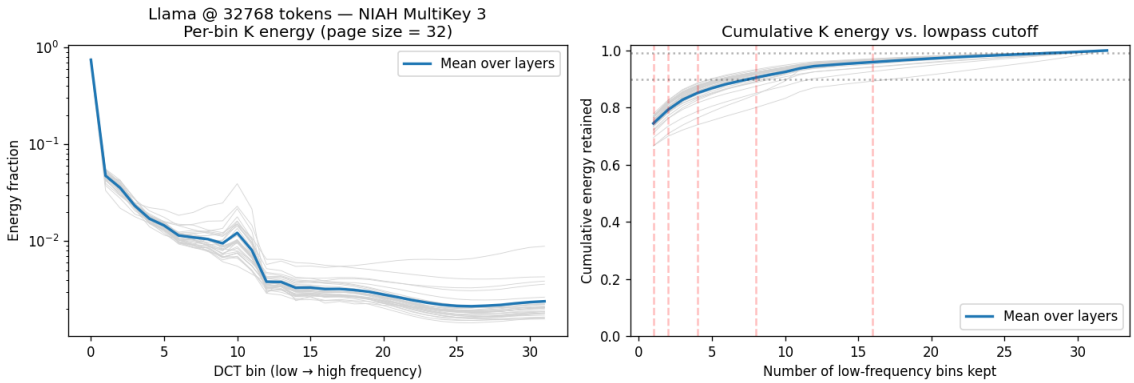

Figure 3.2: Per-bin DCT energy and cumulative energy of decoder-transformer key pages on Llama-3.1-8B-Instruct, page size P=32, RULER NIAH-MultiKey 3 at 32K context. Left: per-bin energy fraction; each thin grey line is a single transformer layer, the thick blue line is the across-layer mean. The spectrum is strongly low-pass. Right: cumulative fraction of energy retained as a function of the lowpass cutoff; the first four bins already cover $\approx 85\%$ of the per-page energy and eight bins cover $\approx 90\%$.

This concentration has a direct consequence for sparse attention. The attention score that determines whether page p should be selected for a query q is the inner product $q^\top K_p[n, \cdot]$ for some token n in the page. If K_p itself lives in a low-rank low-frequency subspace, then a DCT-lowpass reconstruction of K_p should be a near-lossless proxy for the selection score, even after collapsing P tokens into a handful of representative tokens. The next section formalizes this proxy and quantifies how well it routes queries to the right page.

3.3 DCT-lowpass-IDCT Page Proxy

Section 3.2 established that a page of keys $K_p \in \mathbb{R}^{P\times d}$ is concentrated in its leading DCT bins. DCT-Page exploits this by replacing each page with a much shorter sequence that retains only those bins.

Fix a target proxy length C with $1 \le C \le P$ ("compression ratio" C/P). Let $F_P \in \mathbb{R}^{P\times P}$ be the orthonormal DCT-II matrix from Eq. (3.3), let $F_C \in \mathbb{R}^{C\times C}$ be the same construction with P replaced by C, and let $E_C = [\, I_C \;\mid\; 0\,] \in \mathbb{R}^{C\times P}$ select the first C coordinates. The

DCT-lowpass-IDCT page proxy is

$$K_p^{\text{proxy}} \;=\; M\,K_p \quad \text{with} \quad M \;=\; \sqrt{\tfrac{C}{P}}\,F_C^{\top}\,E_C\,F_P \;\in\; \mathbb{R}^{C\times P}, \tag{3.4}$$

i.e. apply the length-P DCT, keep the leading C coefficients, apply the length-C IDCT, and rescale by $\sqrt{C/P}$ so that the C-token proxy matches the per-token energy density of the original P-token page. Crucially M depends only on (P, C), so it is computed once at start-up and cached as a single $C \times P$ matrix; at runtime, proxy construction is a single batched matrix multiply.

Compression knob. The proxy length C trades selection fidelity against proxy-cache size and per-step compression cost. At C=1 the entire page collapses to a single token, which is fast and minimizes memory but discards the higher-frequency structure that distinguishes pages from one another. At C=4 the proxy keeps the four energetic bins identified in Figure 3.2, capturing $\approx 85\%$ of the per-page key energy while still compressing the cache by 8×. The main configuration in this thesis uses C=4 (i.e. a compress_ratio of $1/8 = 0.125$); Section 5.5 reports the sweep over $C \in \{1, 2, 4, 8\}$ that justifies this choice.

Incremental update during decode. A decode step appends one token to the KV cache. Until the currently open page accumulates a full P tokens it lives in the recent zone and is attended in full; only when it closes does it become a new middle page and require its proxy to be computed. Consequently at most one new page closes per decode step, and the proxy cache is updated as

$$K_{\text{new}}^{\text{proxy}} \;=\; M\,K_{\text{new}}, \qquad M \in \mathbb{R}^{C\times P},\; K_{\text{new}} \in \mathbb{R}^{P\times d}. \tag{3.5}$$

This is a single matrix multiply per (batch, kv-head) tuple, costing $O(P \cdot C \cdot d)$ scalar multiplications. The cost is paid *only when a page closes*, i.e. once every P decode steps, so amortized per-step compression work is $O(C \cdot d)$.

After N middle pages have accumulated the proxy cache occupies $O(N \cdot C \cdot d)$ entries per (batch, kv-head). Both the cache size and the per-page compression cost depend only on the number of *closed* middle pages and the page size P, not on the total context length L; the proxy machinery therefore imposes no extra context-length scaling on top of the unavoidable $O(L)$ KV

cache itself.

3.4 Scoring and Top-k Selection

The proxy cache from Section 3.3 feeds the page-scoring step. Let $Q \in \mathbb{R}^{H_q \times d}$ be the decode query (one token, H_q query heads) and let the model use grouped-query attention (GQA) with H_k key-value heads and $G = H_q/H_k$ query heads per group. Index query heads by (h, j) where h is the KV head and $j \in \{0, \ldots, G-1\}$ is the position within the group.

Per-page score. For page p with proxy $K_p^{\text{proxy}} \in \mathbb{R}^{C \times d}$, form the scaled inner products

$$g_{h,j,p,c} \;=\; \frac{1}{\sqrt{d}}\, Q_{h,j}^{\top}\, K_p^{\text{proxy}}[c,:], \qquad c = 0, \ldots, C-1, \tag{3.6}$$

and reduce over the C proxy tokens with a *proxy-token aggregator* $\rho \in \{\max, \text{mean}\}$ and over the G query heads in the GQA group with a *GQA-group aggregator* $\gamma \in \{\max, \text{mean}\}$:

$$S_{h,p} \;=\; \gamma_{j=0,\ldots,G-1}\Big(\rho_{c=0,\ldots,C-1}\, g_{h,j,p,c} \Big). \tag{3.7}$$

$S_{h,p}$ is a single scalar measuring how relevant page p is to kv-head h's decode query. The default configuration is $\rho = \max$ and $\gamma = \max$, which matches a hottest-token / hottest-head heuristic: page p scores high if *any* query head in the group has a large inner product with *any* of its proxy tokens. Section 5.5 reports sensitivity to these choices.

Top-k selection. Let $\mathcal{T}_h \subseteq \{0, \ldots, N+R\}$ denote the set of *page indices* that kv-head h will attend to during the current decode step. DCT-Page constructs $\mathcal{T}_h$ as the union of three pieces:

$$\mathcal{T}_h \;=\; \underbrace{\{0, \ldots, S-1\}}_{\text{sink pages}} \cup \text{TopK}_k\big(S_{h,\cdot}\big) \;\cup\; \underbrace{\{N, \ldots, N+R\}}_{\text{recent + open pages}}, \tag{3.8}$$

where TopK_k returns the k middle-page indices with the largest scores $S_{h,p}$. Note that the top-k set is selected *per kv-head*, so different heads in the same layer may pick different pages. The decode then performs ordinary scaled-dot-product attention against the keys and values

gathered from $\mathcal{T}_h$; the remaining $N - k$ middle pages contribute *nothing* to the output.

To match comparable sparse-attention baselines fairly, we report budgets as the total number of *fixed-size* pages attended per step, denoted B, defined as

$$B \;=\; S + k + R, \tag{3.9}$$

the sum of the sink, selected-middle, and full-recent contributions. The selection rule of Eq. (3.8) requires $k \leq N$ — the top-k operator picks k pages out of the N middle pages currently in the cache (Eq. (3.2)); N grows with the context length L while k is a configuration constant. Setting B=64 with S=1, R=4 therefore fixes k=59, and the per-step fixed-size budget is

$$B \cdot P \;=\; (1 + 59 + 4) \cdot 32 \;=\; 2{,}048 \text{ tokens},$$

regardless of context length, which is the source of DCT-Page's near-constant decode cost above the $L_{\min}$ threshold and the figure used by Chapter 5 to match sparse-attention baselines at an equal selected-token budget. The always-attended open page contributes an additional $0, \ldots, P-1$ tokens per step depending on the current alignment of the cache; this variable contribution is excluded from B.

Hardware-aware top-k. For $N \leq 1024$ pages, DCT-Page performs the entire top-k in a single fused kernel; for longer contexts it falls back to a two-stage partition-then-merge implementation. Both paths use a parallel bitonic sort that fits the per-head working set in shared memory; this is the dominant decode-time cost saving relative to a CPU- or Python-level top-k launch.

3.5 Implementation on a Paged-Attention Runtime

The decode-time forward of DCT-Page can be viewed as a four-stage pipeline composed on top of an existing paged-attention runtime (FlashInfer [4]). Three of the four stages are scoring/selection operations that FlashInfer does not provide; the last stage is the attention itself. Figure 3.3 summarizes the data flow.

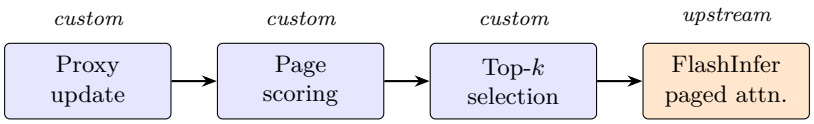

Figure 3.3: DCT-Page decode-time pipeline. Three custom GPU kernels prepare the per-(batch, kv-head) page indices that should be attended; the final attention is dispatched to FlashInfer's paged-attention primitive over those indices.

Shared paged KV layout. DCT-Page reuses FlashInfer's paged KV layout, in which the per-layer KV cache is a pre-allocated tensor of fixed-size pages. This layout naturally aligns with the page abstraction of Section 3.1, so no auxiliary indexing structure is required: a page index p refers to the same physical block of P tokens for both proxy construction and attention.

Custom scoring and selection. The proxy update (Eq. (3.5)), page scoring (Eq. (3.7)), and top-k selection (Eq. (3.8)) each run as a single fused GPU kernel. The output of the pipeline is the per-(batch, kv-head) list of page indices $\mathcal{T}_h$. A pure-PyTorch implementation of each stage is retained for correctness checks and for environments without Triton.

Attention via FlashInfer's paged primitive. Given $\mathcal{T}_h$, the attention itself is dispatched to FlashInfer's variable-length paged-attention primitive. Let $b \in \{0, \ldots, B-1\}$ index the real batch (the input sequences fed to the model) and $h \in \{0, \ldots, H_k - 1\}$ index the kv head; DCT-Page's selection rule (Eq. (3.8)) runs independently per (b, h) pair and therefore produces a distinct page list per pair, which FlashInfer's upstream API does not directly support (it accepts only one page list per batch slot). The runtime sidesteps this by flattening the (b, h) index space into a single "virtual batch" dimension of size $B \cdot H_k$; each slot of this virtual batch carries its own page list, and FlashInfer treats it as an ordinary batched paged-attention call against exactly those pages, without materializing the unselected KV. This is how DCT-Page realizes per-kv-head heterogeneous selection at hardware speed without forking the upstream kernel.

This composition keeps the new code paths small — only the three scoring/selection kernels — while reusing a battle-tested attention primitive for the heavy lifting. Section 5.3 quantifies the resulting throughput.

3.6 Configuration Summary

Table 3.1 lists the default DCT-Page hyperparameters used throughout this thesis; ablations in Section 5.5 sweep (P, B, C, ρ, γ) while holding the rest fixed.

Table 3.1: Default DCT-Page configuration.

Symbol	Meaning	Default
P	Page size (tokens per page)	32
B	Per-step page budget ($S + k + R$)	64
S	Number of sink pages	1
R	Number of full recent pages (excludes the open page)	4
k	Selected middle pages, $= B - S - R$	59
C/P	Compression ratio of the page proxy	1/8
$L_{\min}$	KV length below which decode reverts to dense	8,192
ρ	Proxy-token aggregator	max
γ	GQA-group aggregator	max

Under these defaults, the per-step fixed-size budget evaluates to

$$B \cdot P \;=\; \big(\underbrace{1}_{S} + \underbrace{59}_{k} + \underbrace{4}_{R}\big) \cdot 32 \;=\; 2{,}048 \text{ tokens},$$

which is the selected-token budget used to match sparse-attention baselines in Chapter 5. As noted in Section 3.4, the always-attended open page adds a further $0, \ldots, P-1$ tokens per step.

Chapter 4 Experimental Setup

This chapter fixes the models, benchmarks, sparse-attention baselines, and speed-measurement protocol used throughout Chapter 5 and Chapter 6.

4.1 Models

All experiments use two open-weight, decoder-only language models at the ≈8B-parameter scale: Llama-3.1-8B-Instruct [9] and Qwen3-8B [10]. Both use rotary positional embedding (RoPE) and grouped-query attention (GQA); Qwen3-8B additionally applies QK-Norm to the query and key projections. All experiments run in `bfloat16` on a single NVIDIA RTX A6000 (48 GiB) unless noted otherwise.

4.2 Benchmarks

DCT-Page is evaluated on three long-context benchmark suites. All prompt-formatting follows the chat-template conventions baked into each model's tokenizer, with the exceptions documented in Section 4.2.2.

4.2.1 RULER

RULER [11] is a synthetic long-context test suite of 13 tasks, used here at a default context length of 32K tokens. The tasks fall into four families: needle-in-a-haystack retrieval (NIAH Single 1–3, NIAH MultiKey 1–3, NIAH MultiValue, NIAH MultiQuery), variable tracking (VT), word-frequency extraction (Common Words Extraction, Frequent Words Extraction), and long-document question answering (QA-1, QA-2). Each task is scored by its native exact-match or substring metric.

4.2.2 LongBench v1 and v2

LongBench v1 [12] provides a suite of English long-context tasks across single-document QA, multi-document QA, summarization, and few-shot in-context learning. We evaluate on the six tasks most representative of these categories: NarrativeQA, Qasper, MultiFieldQA-en, 2WikiMultihopQA, GovReport, and TriviaQA. Metrics are the official per-task ones (F1 for QA, ROUGE-L for summarization, classification accuracy for in-context learning). TriviaQA is evaluated without a chat template; the other five tasks use the model's chat format.

LongBench v2 [13] is a 503-question multiple-choice benchmark with four answer options per question. We report overall accuracy and break it down by difficulty (`easy` / `hard`) and by context length (`short` / `medium` / `long`).

4.3 Baselines and Hyperparameters

DCT-Page is compared against five sparse-attention methods organized into two paradigms. All five leave prefill unchanged and modify only decode-time attention; their algorithmic details and provenance are covered in Chapter 2. This section fixes the configuration with which each method is run in Chapter 5.

Page-based baselines. Quest [1], SeerAttention-R [2], and InfLLM [5] share the fixed-page paradigm with DCT-Page. For each we match the *selected-token budget* $B \cdot P = 64 \cdot 32 = 2{,}048$ tokens per decode step, the same budget DCT-Page uses in Section 3.6; settings within that budget follow each method's upstream recommendation. Table 4.1 summarizes the per-method configuration.

Cluster-based reference. Multipole Attention [3] organizes keys into a k-means cluster tree rather than fixed-size pages, and its per-step work cannot be matched against the page-based budget in a one-to-one fashion. Multipole typically achieves the highest accuracy across our benchmarks but at substantially longer decode latency, so we exclude it from the main accuracy tables and instead devote §5.4 to its accuracy–throughput trade-off against DCT-Page. Mul-

tipole is run with a single-level configuration of 3.125% leaf-cluster retention — conceptually matched to DCT-Page's P=32 page granularity — and a per-level token-budget threshold of 2,048. Clusters are formed at the end of prefill, and additional clusters are added during decode as the cache grows.

Full-KV reference. Every accuracy table additionally includes a Full-KV row, which is each model run with its stock dense decode attention. This is the accuracy upper bound that all sparse methods aim to approach at lower decode-time cost.

Model coverage. Page-based baselines are run on whichever model the upstream code supports: SeerAttention-R is reported on Qwen3-8B only (the released gates are trained on the Qwen family, with no equivalent Llama-3 checkpoint), InfLLM on Llama-3.1-8B-Instruct only (the upstream targets the Llama family), and Quest on both models. Multipole and Full-KV are reported on both models.

Table 4.1: Per-method configuration used in the experiments. "Budget" denotes the per-decode-step token budget that the method's selection rule must satisfy.

Method	Key settings	Models
DCT-Page	P=32, B=64, C/P=1/8, ρ=γ= max	both
Quest	page size 32, budget 2,048 tokens	both
SeerAttention-R	token-budget mode, budget 2,048 tokens	Qwen3 only
InfLLM	block size 32, top-64 blocks, sink 32, local 4,096, repr. top-4	Llama only
Multipole	3.125% leaf retention ($\equiv P$=32), threshold 2,048, centroid replacement	both
Full-KV	dense decode attention	both

4.4 Speed-Measurement Protocol

All speed numbers are collected on a single NVIDIA RTX A6000 (48 GiB) from one unified profiler that drives the model with random token inputs and records both the end-to-end decode latency and a per-stage breakdown in a single run. Two execution paths are compared:

- *Full-KV* — the model with its stock dense decode attention, served by upstream FlashInfer's paged-attention primitive.

- *DCT-Page* — the production configuration introduced in Section 3.5, in which DCT-Page's scoring/selection kernels feed per-(batch, kv-head) page indices into the same upstream FlashInfer primitive via a virtual-batching scheme that requires no custom fork of the attention runtime.

CUDA graphs. Every speed measurement reported in this thesis uses CUDA-graph capture for the decode loop. This pins the indices/indptr buffers that the paged-attention primitive reads and removes per-step Python and indices-buffer allocation overheads that would otherwise dominate at short context length.

Throughput measurement. For each (L, batch) cell with $L \in \{8, 16, 32, 64, 128\}$K tokens and batch sizes $\{1, 2\}$, the profiler runs 8 warm-up replays followed by 128 timed decode-step replays of the captured CUDA graph, bracketed by a pair of CUDA events that span the entire timed loop. Per-step wall-clock latency is the elapsed time of this bracket divided by the number of replays, and is reported as two quantities: (i) *attention-only* latency (the time spent inside the attention sub-module, including scoring/selection for DCT-Page), and (ii) *end-to-end* per-step latency (the same chain extended through the MLP, layer-norm, and projection). Speedups in Section 5.3 are reported as the ratio of the Full-KV latency to the DCT-Page latency at the same (L, batch).

Per-stage breakdown. Recording CUDA events between substeps inside the captured graph is not supported on the torch + CUDA build used in this thesis (event records issued during graph capture either error out or fail to produce timings on replay), so the per-stage breakdown of Section 5.3 is collected through the kernel-level path instead. Concretely, the same timed replay loop is wrapped in a `torch.profiler` session that captures CUPTI kernel activity; each kernel launched inside one replay window is then bucketed by its kernel name into one of the eight decode substeps — QKV projection, RoPE + cache append, page-region view (a stride-only operation, no kernel), proxy update, page scoring, fused top-k/KV-assembly, the upstream FlashInfer attention kernel, and the output projection. The bucketing dictionary is anchored to specific kernel symbols emitted by FlashInfer, the DCT-Page Triton kernels, and the upstream

paged-attention primitive; a residual "all-kernel reconciliation" check ensures that the sum of bucketed times agrees with the total CUPTI activity to within sub-millisecond tolerance, so unbucketed kernels (typically a renamed cuBLASLt symbol) cannot silently disappear from the table.

For Full-KV the same bucketing collapses to four populated substeps (QKV, RoPE + cache append, FlashInfer attention, output projection); all other DCT-Page substeps are structurally absent. In both modes, QKV projection and the output projection operate on the single query token and are identical across the two paths, so the reported attention-time budget is the sum of the remaining substeps:

$$t^{\text{attn}}_{\text{Full-KV}} \quad \text{vs.} \quad t^{\text{proxy}}_{\text{DCT-Page}} + t^{\text{score}}_{\text{DCT-Page}} + t^{\text{topk+assemble}}_{\text{DCT-Page}} + t^{\text{attn}}_{\text{DCT-Page}}.$$

Reporting the comparison as a sum keeps the overhead of DCT-Page's bookkeeping kernels charged against its measured speedup.

Chapter 5 Results

5.1 Long-Context Accuracy: RULER

Table 5.1 reports per-task RULER accuracy at 32K context for the two base models, comparing DCT-Page against the page-based sparse-attention baselines. All sparse methods are run at the $B \cdot P = 2{,}048$ selected-token budget fixed in Section 4.3; bold entries mark the best sparse-method score per column.

Table 5.1: RULER accuracy on 13 tasks at 32K context, in percent. Models: Llama = Llama-3.1-8B-Instruct, Qwen3 = Qwen3-8B. Sparse methods are matched at a 2,048-token selected budget. Task abbreviations: S = NIAH Single, MK = NIAH MultiKey, MV = NIAH MultiValue, MQ = NIAH MultiQuery, VT = Variable Tracking, CWE = Common Words Extraction, FWE = Frequent Words Extraction, QA = long-document question answering. Best sparse-method score per column in bold.

Model	Method	S1	S2	S3	MK1	MK2	MK3	MV	MQ	VT	CWE	FWE	QA1	QA2	Avg
Llama	Full-KV	100.00	100.00	100.00	100.00	96.00	100.00	98.00	100.00	98.40	48.40	93.33	72.00	48.00	88.78
	Quest	**100.00**	**100.00**	**100.00**	**100.00**	**96.00**	32.00	97.00	99.00	**97.60**	**30.00**	82.67	**72.00**	**48.00**	81.10
	InfLLM	**100.00**	88.00	**100.00**	84.00	12.00	8.00	75.00	92.00	44.00	6.40	**93.33**	32.00	32.00	58.98
	DCT-Page	**100.00**	**100.00**	**100.00**	**100.00**	**96.00**	**92.00**	**100.00**	**100.00**	96.00	28.00	**93.33**	**72.00**	44.00	**86.26**
Qwen3	Full-KV	100.00	100.00	100.00	96.00	88.00	96.00	95.00	100.00	96.00	86.40	93.33	56.00	48.00	88.83
	Quest	**100.00**	**100.00**	**100.00**	92.00	**88.00**	8.00	90.00	90.00	98.40	**58.80**	84.00	48.00	**48.00**	77.32
	SeerAttention-R	**100.00**	84.00	92.00	92.00	40.00	28.00	65.00	88.00	78.40	21.60	**90.67**	44.00	44.00	66.74
	DCT-Page	**100.00**	**100.00**	**100.00**	**96.00**	**88.00**	**72.00**	**97.00**	**100.00**	**100.00**	46.00	89.33	**52.00**	**48.00**	**83.72**

5.2 Long-Context Accuracy: LongBench v1 and v2

LongBench measures long-context performance on realistic text rather than synthetic retrieval. Table 5.2 reports per-task scores on the six-task v1 subset (Section 4.2.2); Table 5.3 reports v2 multiple-choice accuracy broken down by difficulty (easy / hard) and by context length (short / medium / long).

5.3 Decode Throughput and Per-Stage Profile

This section reports decode-time speedup of DCT-Page over Full-KV at five context lengths $L \in \{8, 16, 32, 64, 128\}$K tokens and two batch sizes (1, 2), measured under the protocol of

Table 5.2: LongBench v1 scores on six representative tasks. Models: Llama = Llama-3.1-8B-Instruct, Qwen3 = Qwen3-8B. Metrics are per-task official metrics (F1 for QA, ROUGE-L for summarization, accuracy for in-context learning). Best sparse-method score per column in bold.

Model	Method	NarrativeQA	Qasper	MFQA-en	2WikiMQA	GovReport	TriviaQA	Avg
Llama	Full-KV	29.24	45.37	54.99	45.33	35.34	91.65	50.32
	Quest	28.99	43.08	51.25	44.80	29.97	91.98	48.35
	InfLLM	25.80	43.89	49.41	**46.47**	34.70	**92.07**	48.72
	DCT-Page	**29.50**	**45.96**	**55.08**	44.68	**35.04**	91.34	**50.27**
Qwen3	Full-KV	24.98	48.32	51.85	43.14	33.46	90.48	48.70
	Quest	**25.47**	45.74	50.74	43.15	31.20	90.30	47.77
	SeerAttention-R	25.30	47.13	50.85	**43.33**	32.54	90.31	48.24
	DCT-Page	24.93	**48.01**	**51.93**	42.89	**33.54**	**90.48**	**48.63**

Table 5.3: LongBench v2 multiple-choice accuracy (%) broken down by difficulty and context-length category. Models: Llama = Llama-3.1-8B-Instruct, Qwen3 = Qwen3-8B. Best sparse-method score per column in bold.

Model	Method	Overall	Easy	Hard	Short	Medium	Long
Llama	Full-KV	29.82	30.21	29.58	35.00	26.05	28.70
	Quest	18.29	22.40	15.76	**38.33**	10.70	0.00
	InfLLM	26.04	25.52	26.37	31.67	**26.05**	16.67
	DCT-Page	**30.02**	**33.33**	**27.97**	34.44	**26.05**	**30.56**
Qwen3	Full-KV	29.64	31.25	28.62	28.70	26.98	33.33
	Quest	18.09	21.88	15.76	38.33	9.77	0.93
	SeerAttention-R	**32.60**	**36.46**	30.23	**38.89**	**30.23**	**26.85**
	DCT-Page	32.01	33.33	**31.19**	**38.89**	28.84	**26.85**

Section 4.4. Figure 5.1 shows the speedup curves; Table 5.4 decomposes the attention-time budget into the per-stage components defined in Section 4.4.

Reading Table 5.4 from the bottom up, DCT-Page's total decode-time attention budget of 1.389 ms decomposes into 0.759 ms of bookkeeping (proxy update, page scoring, and the fused top-k/KV-assembly kernel) and 0.630 ms of attention itself. The bookkeeping is therefore not free — it is comparable in cost to the sparse attention kernel — but it is an order of magnitude cheaper than the 6.400 ms Full-KV attention kernel it displaces. The speedup over Full-KV does not come from the bookkeeping being negligible; it comes from the attention kernel operating on a fixed budget of $B \cdot P = 2{,}048$ tokens regardless of context length. The one stage that genuinely is free is the proxy update itself, at 0.004 ms per step, or 0.3% of the DCT-Page total: this is the payoff of the incremental cache update of Section 3.3, which charges only the freshly closed page each step rather than recomputing proxies for the full cache.

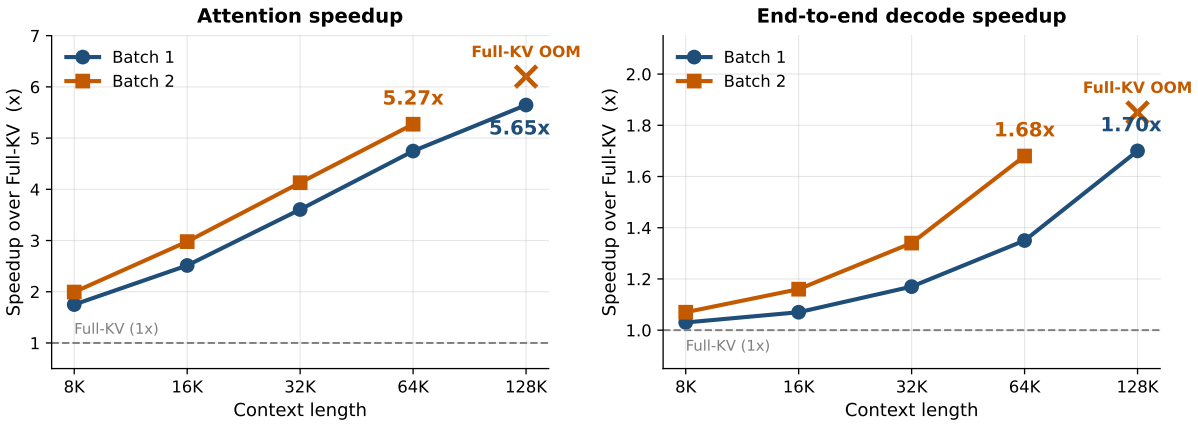

Figure 5.1: Decode speedup of DCT-Page over Full-KV on a single A6000 with Llama-3.1-8B-Instruct. Left: attention-only speedup — up to 5.65× at 128K context (batch 1) and 5.27× at 64K (batch 2). Right: end-to-end decode speedup including MLP and layer-norm — 1.70× at 128K (batch 1) and 1.68× at 64K (batch 2). "Full-KV OOM" marks configurations at which the dense baseline exhausts the 48 GiB device memory.

5.4 Comparison with Cluster-Based Multipole Attention

Multipole Attention is omitted from the head-to-head accuracy tables above because its cluster-based decode path lies on a different point of the accuracy–throughput Pareto frontier than the page-based methods (Section 4.3). Table 5.5 reports DCT-Page and Multipole side by side on each accuracy benchmark and on decode throughput, so that the trade-off can be read directly.

5.5 Ablations

We sweep the four DCT-Page hyperparameters that most influence the accuracy–throughput trade-off, holding all others at the defaults of Table 3.1. All sweeps report RULER average accuracy on Llama-3.1-8B-Instruct at 32K context.

Two observations follow from Table 5.6. First, decode throughput is essentially flat across $P \in \{32, 64, 128\}$ at 39.15–39.37 tok/s, a spread of under 1%; the smallest page size P=16 pays a $\approx$ 5% throughput penalty from doubling the number of pages and the corresponding bookkeeping work, while $P \geq 32$ is essentially constant. Second, accuracy moves in the opposite direction: P=16 leads at 87.47% RULER Avg and accuracy degrades monotonically as P grows,

Table 5.4: Per-stage decode-time attribution at $L = 32{,}768$, batch 1. The Full-KV row reports the time spent inside its attention kernel; the DCT-Page rows split the equivalent attention-time budget into proxy update, page scoring, the fused top-k selection / KV assembly kernel, and the final attention kernel. All values are in milliseconds per decode step, averaged over 128 timed steps. QKV and output projections are identical between the two paths and are excluded.

Stage	Time (ms/step)
Full-KV: attention kernel	6.400
DCT-Page: proxy update	0.004
DCT-Page: page scoring	0.443
DCT-Page: top-k selection + KV assembly	0.312
DCT-Page: attention kernel	0.630
DCT-Page: total	1.389

Table 5.5: DCT-Page vs. Multipole Attention. RULER and LongBench v1 are reported as averages across their respective tasks; LongBench v2 is reported as overall accuracy. Decode throughput is the tokens per second at $L = 32{,}768$, batch 1. Multipole's per-step computation is data-dependent (clusters are re-formed during generation and selected clusters are scattered in memory); the CUDA-graph wrapper required to capture this irregular control- and memory-access pattern was beyond the scope of our implementation, so for an apples-to-apples comparison both methods are measured in eager mode here. The CUDA-graph throughput of DCT-Page is reported in Section 5.3.

	RULER Avg		LongBench v1		LongBench v2		Throughput
Method	Llama	Qwen3	Llama	Qwen3	Llama	Qwen3	(tok/s)
DCT-Page	86.26	83.72	50.27	**48.63**	**30.02**	**32.01**	**26.90**
Multipole	**89.19**	**88.04**	**50.57**	48.48	28.23	31.41	7.50

dropping to 84.10% at P=128. Page granularity is therefore a selection-fidelity knob more than a throughput knob in this regime; P=32 sits at the elbow of the trade-off and is used as the default throughout the thesis.

Table 5.6: Page size P sweep at fixed selected-token budget $B \cdot P = 2{,}048$. Smaller P gives finer selection at the cost of more bookkeeping work; larger P amortizes overhead but loses selectivity inside the page. RULER accuracy and throughput measured on Llama-3.1-8B-Instruct at $L = 32{,}768$, batch 1, in drop mode.

P	B (pages)	RULER Avg (%)	Tok/s @ 32K
16	128	87.47	37.28
32	64	86.07	39.37
64	32	84.69	39.35
128	16	84.10	39.15

Table 5.7: Selected-token budget sweep at fixed P=32. k is the middle-page count $B - S - R$ at S=1, R=4. Llama-3.1-8B-Instruct, $L = 32{,}768$.

Budget (tokens)	B (pages)	k (middle)	RULER Avg (%)
1,024	32	27	83.58
2,048	64	59	86.07
4,096	128	123	87.90

Table 5.8: Compression ratio sweep at fixed P=32, B=64. C is the number of proxy tokens retained per page (Section 3.3). Llama-3.1-8B-Instruct, $L = 32{,}768$.

C	C/P	RULER Avg (%)
1	1/32	79.62
2	1/16	83.07
4	1/8	86.07
8	1/4	87.35

Table 5.9: Choice of proxy-token aggregator ρ and GQA-group aggregator γ (Section 3.4) at fixed P=32, B=64, C=4. The scorer choice (ρ) dominates the GQA aggregator choice (γ); ρ= max outperforms ρ=mean by ≈ 5 points regardless of γ, while the spread between the two ρ= max rows is within RULER's per-run variance. Llama-3.1-8B-Instruct, $L = 32{,}768$.

ρ	γ	RULER Avg (%)
max	max	86.07
max	mean	86.68
mean	max	81.91
mean	mean	81.05

Chapter 6 Analysis

6.1 Why DCT-Page Works: Attention-Mass Recall

Top-k page selection is only as good as the pages it picks. We measure how well a scorer picks them with a single scalar, the *paged attention-mass recall*, which captures the fraction of the dense softmax distribution that lands inside the selected page set.

Definition. Consider a decode step and a single (layer, kv-head) pair. Let $a \in \mathbb{R}^L$ be the dense softmax attention weights the model *would* have produced with the full KV cache, and let $\mathcal{M}$ denote the *selectable* middle pages, excluding the always-attended sink and recent zones. For a scorer that picks k middle pages $\mathcal{T} \subseteq \mathcal{M}$, let $M(\mathcal{T}) = \sum_{p \in \mathcal{T}} \sum_{n \in \text{page}(p)} a_n$ be the dense softmax mass it captures, and let

$$M^\star \;=\; \max_{\mathcal{T}' \subseteq \mathcal{M},\, |\mathcal{T}'|=k} M(\mathcal{T}')$$

denote the maximum mass any size-k middle-page selection could capture — the "mass-topk" oracle that would have picked the heaviest middle pages had it known a in advance. The paged mass-recall ratio is

$$m(\mathcal{T}) \;=\; \frac{M(\mathcal{T})}{M^\star} \;\in\; [0,1]. \tag{6.1}$$

Both numerator and denominator strip the always-attended sink and recent floor, so $m(\mathcal{T})$ isolates how well a scorer ranks the selectable middle pages relative to the achievable ceiling. $m(\mathcal{T}) = 1$ means the scorer matches the mass-topk oracle on the middle pages; small values mean it routed the query away from the middle pages the dense softmax actually weighted heavily. Any scorer that achieves high $m(\mathcal{T})$ at a fixed budget is in a strong position to approximate Full-KV regardless of its internal mechanism.

Measurement. We compute $m(\mathcal{T})$ by running each model in its standard Full-KV decode mode on a RULER NIAH-MultiKey 3 trace at $L = 32{,}768$ and Llama-3.1-8B-Instruct, observing

the dense attention weights at every layer and step, and asking each scorer — DCT-Page's DCT-lowpass-IDCT proxy and Quest's per-channel min/max statistics — to produce a page set $\mathcal{T}$ at the same selected-token budget $B \cdot P = 2{,}048$. Equation (6.1) is then evaluated on the dense weights. Figure 6.1 reports the result at $P = 16$, $B = 128$, sweeping the DCT-proxy length $c \in \{1, 2, \ldots, 16\}$ for the DCT-Page scorer; Quest's per-channel min/max scorer has no analogous knob and appears as a single horizontal line. The figure plots the per-layer mean and across-layer $\pm 1\sigma$ band for each scorer, together with the trivial $m \equiv 1$ ceiling.

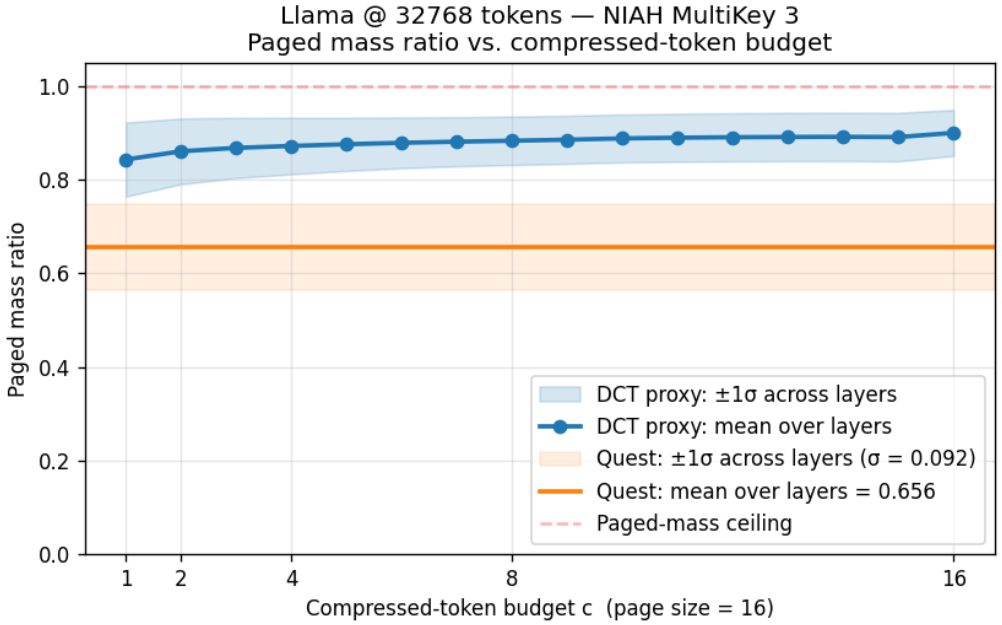

Figure 6.1: Paged mass-recall ratio (Eq. (6.1), i.e. the fraction of the mass-topk oracle's achievable middle-page mass that a scorer captures at the same size-k budget) on a RULER NIAH-MultiKey 3 trace at $L = 32{,}768$ with Llama-3.1-8B-Instruct, page size P=16, selected-token budget $B \cdot P = 2{,}048$. The DCT-lowpass-IDCT proxy of Section 3.3 (blue) already reaches $\approx 84\%$ of the oracle at the most aggressive setting c=1 (one proxy token per 16 keys), and the curve climbs gently to $\approx 90\%$ at c=16 before saturating; the per-channel min/max scorer of Quest (orange) reaches only 65.6% at the same budget, regardless of c. Shaded bands are $\pm 1\sigma$ across transformer layers.

Quantitative comparison. At the most aggressive compression setting visible in Figure 6.1 — one proxy token per 16-key page, i.e. a 1/16 compression ratio — the DCT-lowpass-IDCT proxy already reaches $\approx$ 84% of the mass-topk oracle's middle-page recall (averaged across

layers), while Quest's per-channel min/max statistics reach only 65.6% at the same selected-token budget. This is an $\approx$ 18-point gap in the only intermediate quantity that connects scorer geometry to downstream attention error, and it holds before DCT-Page is given any of its bin-budget headroom: as c grows toward P, the DCT curve climbs to $\approx$ 90%, widening the gap to Quest by another $\approx$ 6 points while Quest's line does not move. The DCT-Page curve is also remarkably flat once it clears its first few bins, consistent with the spectral concentration documented in Section 3.2 — the energetic structure of a page already lives in its first few DCT bins, and additional bins mostly fit noise. The flatness justifies operating at small c in production (we use the equivalent $C/P = 1/8$ at P=32 for the default configuration of Section 3.6): it minimizes proxy-cache footprint and per-page compression cost without giving up recall.

Implications for downstream accuracy. The sparse attention output is a softmax-weighted sum of values, so the more achievable middle-page mass a scorer fails to capture, the more dense weight it routes *away* from its selected pages and the larger its deviation from Full-KV at that step. DCT-Page leaves $\approx$ 16% of the oracle-achievable mass uncovered at c=1; Quest leaves $\approx$ 34% uncovered. This $\approx$ 18-point gap is the mechanism behind the +5.16 and +6.40-point RULER averages of DCT-Page over Quest on Llama and Qwen3 reported in Table 5.1. In particular, the largest per-task gap — NIAH-MultiKey 3, where DCT-Page reaches 92.00 on Llama versus Quest's 32.00, and 72.00 on Qwen3 versus Quest's 8.00 — is exactly the task whose dense softmax is the most concentrated on a single page (the needle's page). When the dense distribution is concentrated, leaving 34% of the oracle's mass on the table is much more likely to also miss the needle page entirely than leaving 16%, which is precisely what the per-task numbers show.

6.2 Where DCT-Page Wins and Where It Lags

The headline averages of Chapter 5 hide a wide per-task spread. Reading the rows of Tables 5.1, 5.2, and 5.3 side by side reveals two complementary regimes for page-level selection — one in which the mass-recall advantage of Section 6.1 translates almost directly into accuracy, and one

in which no fixed-budget page selector, DCT-Page included, can fully cover what the task needs. This section walks through the four most informative findings.

6.2.1 Wins on Retrieval-Style Tasks

The retrieval rows of Table 5.1 — NIAH MultiKey, MultiValue, MultiQuery, and Variable Tracking — are where the mass-recall advantage of Figure 6.1 cashes out most directly. On Llama-3.1-8B-Instruct, DCT-Page reaches 92.00 on NIAH-MultiKey 3 against Quest's 32.00 (a 60.00-point gap), 100.00 on NIAH-MultiValue against Quest's 97.00, and 100.00 on NIAH-MultiQuery against Quest's 99.00. The Qwen3-8B picture is even sharper: DCT-Page reaches 72.00 on NIAH-MultiKey 3 against Quest's 8.00 (a 64.00-point gap), 97.00 on NIAH-MultiValue against Quest's 90.00, and 100.00 on Variable Tracking against Quest's 98.40. On the easier retrieval tasks (NIAH-Single 1–3, MultiKey 1–2, MultiQuery) DCT-Page matches Full-KV exactly on both models; the residual gap on the remaining retrieval rows concentrates on NIAH-MultiKey 3.

These tasks share a common structure: the answer lives at a small number of identifiable positions, and the dense softmax at the decode step concentrates sharply on the page(s) holding those positions. Page selection therefore reduces to "find the needle's page," which is exactly what a high-fidelity intermediate representation of the page should support. The DCT-lowpass-IDCT proxy of Section 3.3 keeps the position-correlated low-frequency structure of the page and discards only the high-frequency residual, so the inner product $q^\top K_p^{\text{proxy}}[c,:]$ remains close to the maximum dense score $\max_n q^\top K_p[n,:]$ that would have decided whether the page contains the needle. Quest's per-channel min/max captures only the marginal extreme value of each feature dimension and discards the joint structure across positions inside the page; this is enough for the easier NIAH variants (single-needle MultiKey 1 and 2) but breaks down on MultiKey 3, which is the hardest needle-in-needle setting in the suite.

6.2.2 Lags on Word-Frequency and Aggregation Tasks

The Common Words Extraction (CWE) and Frequent Words Extraction (FWE) rows of Table 5.1 tell the opposite story. On Llama, DCT-Page records 28.00 on CWE against Quest's 30.00 and

Full-KV's 48.40, and 93.33 on FWE (tied with InfLLM, ahead of Quest's 82.67). On Qwen3 the gap widens: DCT-Page records 46.00 on CWE against Quest's 58.80 and Full-KV's 86.40, a 12.80-point loss to Quest and a 40.40-point loss to Full-KV. CWE/FWE are aggregation tasks — the model must count words across the entire context and report the most common — rather than retrieval tasks.

The dense softmax for an aggregation question is by construction *not* concentrated on a single page; the relevant evidence is spread across many pages, and the answer is a function of the broad distribution rather than of any single peak. A fixed-budget page selector restricted to B=64 pages has no way to integrate the contributions of the $N - k$ unselected middle pages, and the per-step error is bounded below by the unrecovered mass however well the scorer ranks pages. This is a *structural* limitation of fixed-budget page selection rather than a deficiency of the DCT proxy specifically, and it suggests a direction — combining page-level top-k with a coarse summary of the unselected mass — that we leave to future work.

6.2.3 Catastrophic Long-Context Failure of Quest

The most dramatic failure mode in Chapter 5 is Quest's behaviour on the Long context bucket of LongBench v2 (Table 5.3). Quest scores 0.00 on Llama and 0.93 on Qwen3 — effectively random performance below the four-choice chance level of 25%. DCT-Page, run with the same 2,048-token selected budget on the same prompts, scores 30.56 on Llama and 26.85 on Qwen3, both within 3 points of Full-KV (28.70 and 33.33 respectively).

We attribute the gap to a structural property of the per-channel min/max scorer: each channel's contribution is a loose upper bound on $q^\top k_n$ computed independently of all other channels. Two pages whose per-channel min/max happen to be similar in the channels where $|q[c]|$ is large — the channels that drive the sum — therefore receive similar overall scores, regardless of how the rest of their internal token structure differs. As context grows with N, the chance of such partial coincidence between an answer-bearing page and an unrelated neighbour grows, and selection becomes increasingly noisy. On LongBench v2 prompts at the Long bucket — which exceed 32K tokens of real text rather than synthetic needles — this effect hits hard, and selection essentially stops working. The DCT-lowpass-IDCT proxy is by construction sensitive

to the position-correlated low-frequency structure inside each page (Section 3.2), so two pages with different internal token structure produce different proxy scores even when their marginal extremes happen to align.

6.2.4 Sparse Attention Exceeding Full-KV on LongBench v2

A more subtle phenomenon visible in Table 5.3 is that DCT-Page slightly exceeds its own Full-KV baseline on overall LongBench v2 accuracy: 30.02 vs. 29.82 on Llama (+0.20) and 32.01 vs. 29.64 on Qwen3 (+2.37). SeerAttention-R on Qwen3 shows the same direction at a larger magnitude (32.60 vs. 29.64). A sparse attention method outperforming its own dense counterpart on a real long-context multiple-choice benchmark is not what we expected to see, and we do not have a clean explanation; one plausible interpretation is an implicit-denoising effect.

By zeroing out the contribution of the $N - k$ unselected middle pages, page-level top-k selection effectively sharpens the softmax toward the highest-scoring evidence and suppresses the long tail of low-mass distractor pages that LongBench v2 questions are specifically designed to include. If a fraction of the dense softmax mass at long context routes to noise rather than to useful evidence, then removing it can plausibly improve the model's discrimination among the four multiple-choice options without changing what the model knows. We report the result as observed: page-level sparsification at the 2,048-token budget does not hurt LongBench v2 overall accuracy relative to Full-KV, and on Qwen3 it consistently helps by a small margin.

6.3 Limitations

Residual gap to Full-KV. DCT-Page is the best page-based method on every aggregate metric reported in Chapter 5, but it is not universally best. On Qwen3-8B RULER it trails Full-KV by 5.11 points (83.72 vs. 88.83, Table 5.1), and on Qwen3-8B LongBench v2 overall accuracy SeerAttention-R edges it out by 0.59 points (32.60 vs. 32.01, Table 5.3). The RULER gap is concentrated in the aggregation tasks discussed in Section 6.2.2, while the SeerAttention-R win is small enough to fall within the statistical envelope noted in Section 6.2.4; nevertheless, the existence of a gap to dense attention and the existence of a method that beats DCT-Page on one benchmark cell are both worth stating plainly.

Decode-only sparsification. DCT-Page modifies only the decode path; prefill remains standard dense attention (Chapter 3). The speedups reported in Section 5.3 therefore apply only to the autoregressive generation phase, and workloads dominated by long-prompt prefill see no benefit from the proxy machinery. This is a deliberate scope choice (the page proxy is most informative once a query is available to score it against) but it should be read as a limitation when comparing to methods that target the prefill side as well.

Evaluation scope. The empirical claims of this thesis rest on two open-weight 8B-parameter models (Llama-3.1-8B-Instruct and Qwen3-8B, Section 4.1) evaluated on a single GPU class (NVIDIA RTX A6000 with 48 GiB). Whether the mass-recall advantage of Section 6.1 transfers to larger models, to non-instruction-tuned bases, or to GPUs with different memory and bandwidth characteristics is an open question. In particular the relative weight of the bookkeeping kernels in Table 5.4 will shift with the ratio of attention to MLP work, so the end-to-end speedup figures should be read as illustrative for the 8B-class regime rather than as a universal constant.

Structural ceiling on aggregation tasks. The shortfall on word-frequency tasks analyzed in Section 6.2.2 is not addressable by tuning the DCT proxy alone; it follows from the fixed-budget page-selection paradigm itself. Any selector that commits all its budget to B pages cannot represent the long tail of mass that aggregation tasks distribute across the remaining $N - k$ middle pages. Closing this gap likely requires either a coarse summary of the unselected pages that contributes to the attention output alongside the top-k selection, or a hybrid scheme that combines page-level top-k with a sparser token-level fallback; both are natural next steps that we do not evaluate here.

Chapter 7 Conclusion

This thesis introduced DCT-Page, a training-free decode-time sparse attention method built on a single structural observation: pages of attention keys live overwhelmingly in their lowest few DCT bins, and a precomputed DCT-lowpass-IDCT projection therefore yields a faithful per-page summary at almost no runtime cost. The resulting system divides the KV cache into fixed-size pages, scores each page by the query's inner product against its low-frequency proxy, and reads only the top-k pages at full precision. The scoring and selection machinery runs as three fused GPU kernels on top of upstream FlashInfer's paged-attention primitive, and the entire decode loop is CUDA-graph capturable.

Empirically, DCT-Page is the strongest page-based sparse attention method in our evaluation. On RULER at 32K context with a matched 2,048-token selected budget, it leads the best page-based baseline by +5.16 points on Llama-3.1-8B-Instruct and +6.40 points on Qwen3-8B, and stays within 0.07 points of Full-KV on LongBench v1. On LongBench v2 it is the only sparse method whose accuracy does not collapse on the longest context bucket — where Quest's per-page min/max scorer drops to 0.93 and below, DCT-Page maintains performance on par with Full-KV. The attention-mass-recall analysis in Section 6.1 traces this advantage to its mechanistic source: at the same per-step budget, DCT-Page reaches $\approx 84\%$ of the mass-topk oracle's achievable middle-page recall against Quest's 65.6%. On a single A6000 GPU, the resulting throughput is 5.65× higher than Full-KV in the attention kernel and 1.70× higher end-to-end at 128K context, and 3.59× higher than the cluster-based Multipole Attention at 32K under matched eager-mode measurement.

Several directions remain open. First, DCT-Page's accuracy gap to Full-KV on Qwen3 RULER (-5.11 points on average) is larger than the corresponding Llama gap, and closing it is the most concrete next step — candidate ideas include richer per-page proxies (more DCT bins, or hybrid scoring that combines DCT with a Quest-style boundary statistic) and per-layer rather than per-model hyperparameter tuning. Second, the method as presented modifies only the decode path; long-prefill workloads currently see no speedup, and extending the frequency-domain summary to the prefill stage — in the spirit of the concurrent FreqKV system [8] —

could yield a unified prefill-and-decode design. Third, our evaluation is restricted to two 8B-class models on one GPU architecture; the behavior of DCT-Page at the 70B+ scale and on non-A6000 hardware (especially memory-bound mobile or edge accelerators) is open. Finally, the aggregation tasks of Section 6.2.2 expose a structural limit of any fixed-budget page selector, and combining page-level selection with a sparse token-level fallback for such tasks is a promising direction that we leave for future work.

Bibliography

[1] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context LLM inference," in *Proceedings of the 41st International Conference on Machine Learning* (*ICML*), 2024. arXiv: 2406.10774 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2406.10774

[2] Y. Gao et al., *SeerAttention-R: Sparse attention adaptation for long reasoning*, 2025. arXiv: 2506.08889 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2506.08889

[3] C. Hooper et al., "Multipole attention for efficient long context reasoning," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2025. arXiv: 2506.13059 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2506.13059

[4] Z. Ye et al., "FlashInfer: Efficient and customizable attention engine for LLM inference serving," in *Proceedings of Machine Learning and Systems* (*MLSys*), 2025. arXiv: 2501.01005 [cs.DC]. [Online]. Available: https://arxiv.org/abs/2501.01005

[5] C. Xiao et al., "InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2024. arXiv: 2402.04617 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2402.04617

[6] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2017. arXiv: 1706.03762 [cs.CL]. [Online]. Available: https://arxiv.org/abs/1706.03762

[7] K. R. Rao and P. Yip, *Discrete Cosine Transform: Algorithms, Advantages, Applications*. San Diego, CA: Academic Press, 1990, ISBN: 978-0-12-580203-1.

[8] J. Kai et al., "FreqKV: Key-value compression in frequency domain for context window extension," in *International Conference on Learning Representations* (*ICLR*), 2026. arXiv: 2505.00570 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2505.00570

[9] A. Grattafiori et al., *The Llama 3 herd of models*, 2024. arXiv: 2407.21783 [cs.AI]. [Online]. Available: https://arxiv.org/abs/2407.21783

[10] A. Yang et al., *Qwen3 technical report*, 2025. arXiv: 2505.09388 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2505.09388

[11] C.-P. Hsieh et al., "RULER: What's the real context size of your long-context language models?" In *Proceedings of the Conference on Language Modeling* (*COLM*), 2024. arXiv: 2404.06654 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2404.06654

[12] Y. Bai et al., "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (*ACL*), 2024. arXiv: 2308.14508 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2308.14508

[13] Y. Bai et al., "LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics* (*ACL*), 2025. arXiv: 2412.15204 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2412.15204

초 록

장문맥 디코더 트랜스포머 언어모델은 디코딩 시간의 대부분을 점점 커지는 KV 캐시를 메모리에서 읽어들이는 데에 사용하며, 결과적으로 스텝당 지연 시간이 HBM 대역폭 한계에 직접적으로 묶이게 된다. Sparse attention 기법은 과거 토큰 중 일부만 선택적으로 읽어 이 병목을 우회하지만, 기존의 페이지 기반 선택 방식은 페이지당 거친 통계량(Quest [1]의 채널별 min/max)이나 모델별로 별도 학습이 필요한 게이트(SeerAttention-R [2])에 의존하며, 클러스터 기반 대안(Multipole Attention [3])은 정확도는 더 높지만 입력 의존적인 디코드 경로의 불규칙한 메모리 접근 패턴 탓에 실측 속도가 페이지 기반 방식보다 수 배 느리다.

본 논문은 DCT-Page를 제안한다. DCT-Page는 학습이 필요 없는 디코드 시점 Sparse attention 기법으로, 다음의 구조적 관찰에 기반한다: 어텐션 키들의 연속된 페이지는 이산 코사인 변환(DCT)의 가장 낮은 몇 개의 주파수 빈에 대부분의 에너지가 집중되어 있으며, 32 토큰 페이지에서 처음 4개의 빈이 전체 페이지 에너지의 평균 약 85% 를 차지한다. DCT-Page는 이 관찰을 활용하여 각 페이지를 짧은 DCT-Lowpass-IDCT 압축값(미리 계산된 선형 사영을 페이지가 닫힐 때마다 한 번의 작은 행렬 곱으로 계산)으로 대체하고, 쿼리와 압축값 토큰들의 내적을 통해 페이지에 점수를 매긴다. 상위 k 개의 페이지만 원본 정밀도로 읽고 나머지는 폐기한다. 점수 계산과 선택은 공식 FlashInfer [4]의 페이지 어텐션 위에서 세 개의 융합된 GPU 커널로 동작하며, 전체 디코드 루프는 CUDA 그래프로 캡처 가능하다.

32K 컨텍스트의 RULER에서 DCT-Page는 가장 강한 페이지 기반 베이스 라인보다 Llama-3.1-8B-Instruct에서 +5.16 점, Qwen3-8B에서 +6.40 점 앞서며, LongBench v1에서 Full-KV와의 차이가 0.07 점 이내, LongBench v2의 가장 긴 컨텍스트 구간에서 정확도가 붕괴하지 않는 유일한 Sparse attention 기법이다. 어텐션 질량 회복(attention-mass recall) 분석은 이 우위의 메커니즘을 정량적으로 설명한다: 동일한 스텝당 예산에서 DCT-Page는 mass-topk 오라클이 중간 페이지에서 얻을 수 있는 최대 질량의 $\approx 84\%$ 에 도달하는 반면 페이지 기반 베이스라인은 65.6% 에 그친다. 단일 A6000 GPU의 128K 컨텍스트에서 DCT-Page는 어텐션 커널 기준 5.65×, end-to-end 기준 1.70× Full-KV보다 빠르다.

주요어: 장문맥 언어모델, 희소 어텐션, KV 캐시, 이산 코사인 변환, 페이지 어텐션